2026 생성형AI활용 프로젝트 · 중간보고

# 근거로 확인하는 AI 변경 요약

기획서에서 실제로 동작하는 프로그램으로

팀프로젝트_최종.zip → 팀프로젝트_진짜최종.zip → **근거 있는 변경 리포트**

**팀 근성**

2026. 11. 9.

기획 발표(10/26) 이후 2주

# 중간보고 목표였던 핵심 기능을 완성했습니다

**v0.5**
현재 버전. 실제 Gemini로 요약까지 동작

**100%**
시연 세트에서 변경 분류가 정답과 일치

**58/58**
AI 문장이 모두 근거 대조 통과 (시연)

**0.97+**
오픈소스 4개 구간 Git 기준 분류 F1

비교 → AI 요약 → 근거 검증 → 리포트 + 웹 화면까지 완성
남은 2주는 사용자 테스트와 품질 개선에 집중합니다.

# 팀 근성

## 지용준

**조장 · 개발 총괄**

- 기획 · 전체 구조 설계
- 비교 엔진 · 문서 추출
- AI 요약 · 근거 검증
- 리포트 · 웹 화면
- 평가 실험 · 문서 작성

## 나현준

**발표**

기획 발표 진행

## 박민제

**데이터 제공**

테스트용 프로젝트·문서 파일 제공

## 최림정

**발표 · 검수**

중간 발표 진행, 리포트 결과 확인

## 신동윤

**발표 · 보조**

최종 발표 진행, 자료 조사

# 버전은 쌓이는데, 무엇이 바뀌었는지는 알기 어렵다

과제.zip → 최종.zip → 최종수정.zip → 진짜최종.zip

### Git (전문가용 도구)

정확하지만 배우기 어려워 입문자는 잘 안 씀

### 파일 비교 도구

한 줄씩 차이만 보여 줘서 전체 그림이 안 보임

### AI 요약

읽기 쉽지만 근거가 없고,
없는 변경을 지어내기도 함

필요한 것: AI 요약의 편리함 + 직접 확인할 수 있는 근거

1. 프로젝트 개요 · 목표 및 범위

# 무엇을, 어디까지, 어떤 형태로

## 주요 기능

- 폴더·zip 두 개의 변경 자동 분류 (추가 / 삭제 / 수정 / 이동)
- AI 계층 요약 (파일 → 폴더 → 전체)
- 문장마다 근거 연결 + 자동 검증
- 개발자용 / 비개발자용 리포트

## 범위

**지원** · 코드·txt·md, hwp·hwpx, docx·pptx·xlsx 본문

**변경 여부만** · 이미지, 스캔 pdf, 비밀 파일

**제외** · 서식 변경, 모델 학습

## 산출물 형태

- Python 프로그램 (명령어)
- 웹 업로드 화면
- HTML 리포트 한 장
- 평가 도구 2종
- 설치 / 실행 안내서

**기대 성과** · 변경 확인 시간 단축 · 비개발 팀원도 진행 상황 파악 · AI 요약을 근거로 검토

1. 프로젝트 개요 · 주제 관련 기술 · 기술 스택

# LLM을 학습시키지 않고, 설계로 믿을 수 있게

### LLM · Gemini Flash
무료 티어로 사용. 변경의 의미를 설명하는 문장만 작성

### API · 구조화 출력
google-genai SDK로 호출, 정해진 JSON 형식으로만 응답

### 프롬프트 설계
근거 번호 필수 · 숫자 금지 · 코드 이름은 백틱 · 추측 금지

### 신뢰성 기법
계층 요약 · 근거 검증 규칙 6개 · 자동 모델 전환

**핵심 원칙**
사실(숫자 / 파일 / 위치)은 프로그램이, 해석(의미)은 AI가 쓴다

**기술 스택** · Python 3.12 · difflib · olefile · google-genai · Streamlit · VS Code · Git · 별도 서버 없이 개인 PC에서 실행 — 모두 무료, GPU 불필요

# 사용자 입장에서 정의한 기능 9가지

| ID | 사용자는 … 할 수 있다 | 상태 |
|---|---|---|
| FR1 | 폴더 또는 zip 두 개를 넣어 비교한다 (한글 zip 파일명 자동 복원) | 완료 |
| FR2 | 캐시 / 빌드 파일은 자동 제외되고, 비밀 파일은 내용이 보호된다 | 완료 |
| FR3 | 추가 / 삭제 / 수정 / 이동과 폴더 이름 변경을 한눈에 본다 | 완료 |
| FR4 | hwp·docx 등 문서 파일도 본문 기준으로 비교한다 | 완료 |
| FR5 | 파일 → 폴더 → 전체 순서의 AI 요약을 읽는다 | 완료 |
| FR6 | 요약 문장의 근거를 눌러 실제 바뀐 줄로 이동한다 | 완료 |
| FR7 | 근거와 맞지 않는 문장에 붙은 ⚠ 표시와 사유를 본다 | 완료 |
| FR8 | 개발자용 / 비개발자용 보기를 전환하고 리포트를 내려받는다 | 완료 |
| FR9 | 파일 하나끼리(예: txt ↔ txt) 직접 비교한다 | 예정 |

# 요구사항과 실제 확인 결과

## 성능

요구 : 비교·검증은 즉시, AI 포함 수 분 이내

**확인** · 비교·검증 1초 미만,
AI 요약은 파일 수에 따라 수 분

## 안정성

요구 : AI 서버 오류에도 리포트가 끝까지 나올 것

**확인** · 재시도·자동 모델 전환·하위 요약 대체.
시연에서 모델 전환이 실제로 동작

## 보안

요구 : 비밀 정보가 외부 AI로 나가지 않을 것

**확인** · .env·키 파일은 내용을 읽지 않고 해시로만 비교

## 확장성

요구 : AI 모델과 파일 형식을 쉽게 바꿀 수 있을 것

**확인** · AI 연결부 분리 → Gemini / Groq / Ollama 설정만으로 교체

2. 요구사항 분석 · 타깃 사용자

# 누가, 언제 쓰는가

### 입문 개발자

Git이 익숙하지 않은 학생 / 소규모 팀
버전 사이에 무엇이 바뀌었는지 빨리 알고 싶음

### 비개발 팀원

기획 / 디자인 담당.
코드를 몰라도 진행 상황을 이해하고 싶음

### 검토자

조교·멘토.
버전 사이의 변화를 근거와 함께 확인하고 싶음

**사용 시나리오**

회의 전날, 팀장이 **최종.zip**과 **진짜최종.zip**을 올려 리포트를 단톡방에 공유
→ 기획 담당은 비개발자용 화면으로 진행 상황 확인 → 의심 가는 설명은 근거를 눌러 직접 확인

# AI는 다섯 단계 중 한 곳에만 들어간다

**웹 화면** app.py · zip 끌어다 놓기

**명령어** cli.py · 폴더·zip 지정

**① 입력**
`loader`
zip / 폴더 읽기,
노이즈 제외,
문서 텍스트 추출

→

**② 비교 엔진**
`compare`
변경 분류, 이동 감지,
변경 조각 번호, 통계

→

**③ AI 요약**
`summarize`
파일 → 폴더 →
전체 → 비개발자용 문장

→

**④ 근거 검증**
`verify`
AI 문장을 근거와 대조
규칙 E1~E6

→

**⑤ 리포트**
`report`
HTML 한 장
개발자용 / 비개발자용

↕

**Gemini API**

프로그램 (항상 같은 결과)  AI (해석 문장만 작성)  + 평가 도구 evaluate · gitcheck, 연결 점검 check_gemini

3. 시스템 설계 · 데이터 흐름

# 단계마다 무엇이 들어가고 무엇이 나오는가

| 단계 | 입력 | 출력 | 담당 | AI 전송 |
|---|---|---|---|---|
| ① 입력 | zip·폴더 두 개 | 파일 목록, 해시, 본문 텍스트 | 프로그램 | 없음 |
| ② 비교 | 파일 목록 두 개 | 변경 목록, 변경 조각 H1…Hn, 통계 | 프로그램 | 없음 |
| ③ AI 요약 | 변경 조각 내용 (숫자 통계 제외) | 요약 문장 S·D·P·N + 근거 번호 | AI | 변경 조각만 |
| ④ 검증 | 요약 문장 + 변경 조각 | 문장별 경고 (E1~E6) | 프로그램 | 없음 |
| ⑤ 리포트 | ①~④의 결과 전부 | HTML 리포트, JSON(선택) | 프로그램 | 없음 |

비밀 파일·노이즈는 ①에서 걸러져 AI로 가지 않음

AI 설명마다 근거가 연결되어 파일 → 폴더 → 전체로 이어짐

# DB 없이, 파일 하나에 정리되는 데이터 구조

**올린 버전 Snapshot**
버전 이름, 자동으로 뺀 파일 목록

1:N

**파일 FileEntry**
경로, 크기, 지문(해시), 종류, 본문

**변경 Change**
추가·수정·삭제·이동, 이전·이후 위치, 줄 수

1:N

**바뀐 부분 Hunk**
근거 번호, 위치, 바뀐 줄

근거

**AI 설명 SummaryItem**
설명 문장, 연결된 근거, 누가 썼는지

**경고 Warning**
맞지 않는 이유 (설명 하나에 여러 개)

결과를 쌓아 둘 필요가 없어 DB 대신 **리포트 파일(HTML·JSON)**로 남김. 문장마다 프로그램이 썼는지 AI가 썼는지 기록

# Gemini API 연동과 오류 대응

### 요청 설정

- google-genai SDK, 무료 티어 Flash 모델
- 응답 형식: JSON 고정 (문장 + 근거 번호)
- '생각' 단계 최소화로 응답 속도 확보
- 시간 제한 120초, 요청 간격 4초
- API 키는 .env 파일에만 보관

| 오류 | 프로그램의 대응 |
|---|---|
| 429 요청 한도 | 5·10·20초 기다린 뒤 재시도 |
| 504 · 5xx 서버 지연 | 재시도, 계속되면 다른 모델로 |
| 404 · 무료 한도 0 | 쓸 수 있는 Flash 모델로 자동 전환 |
| JSON 형식 오류 | 형식을 강조해 한 번 더 요청 |
| API 키 오류 | 즉시 중단하고 해결 방법 안내 |
| 끝내 실패 | 하위 요약으로 대체, '오류'로 표시 |

대체 연동 · Groq(무료 클라우드), Ollama(내 컴퓨터) — OpenAI 호환 방식, 설정만 바꾸면 교체

# 두 사람이 같은 리포트를 다르게 읽는다

**페르소나 A**

## 팀장 · 개발 담당 (3학년)

**상황** · 팀원 코드가 zip으로 모이지만 무엇이 바뀌었는지 매번 열어 봐야 함

**원하는 것** · 바뀐 파일과 줄을 빠르게, AI 요약은 근거로 확인

**쓰는 화면** · 개발자용 · 근거 칩 · diff

**페르소나 B**

## 기획 담당 · 비개발자 (2학년)

**상황** · 코드를 몰라 진행 상황을 팀장에게 매번 물어봄

**원하는 것** · 기능·화면 관점의 쉬운 설명

**쓰는 화면** · 비개발자용 · 사실 영역 숫자

**설계 원칙** · 사실과 AI 문장을 눈으로 구분 · 모든 AI 문장에서 근거까지 한 번 클릭 · 같은 리포트 안에서 보기만 전환

# 화면은 두 개, 결과는 세 가지 보기로

### ① 올리는 화면

- 이전 버전 zip
- 이후 버전 zip
- 설정 (사용할 AI 등)
- 비교하기 버튼
- 진행 상황
- 리포트 내려받기

→

### ② 결과 리포트

**한눈에 숫자**

몇 개가 바뀌었나
폴더가 어떻게 바뀌었나
뺀 파일 목록

**개발자용**

전체 요약
→ 폴더별
→ 파일별
→ 실제 바뀐 줄

**비개발자용**

쉬운 말 요약
→ 근거 보기
→ 개발자용 화면으로

어느 보기에서든 버튼 한 번으로 '실제로 바뀐 부분'까지 내려갈 수 있음

4. UI/UX 설계 · 사용자 흐름 — 실제로 써 보기

# 파일 두 개 올리고, 버튼 한 번

① 올리기 | ② 한눈에 보기 | ③ 근거 확인 | ④ 쉬운 말 보기 | ⑤ 경고 확인




**1** 예전 버전 zip 올리기

**2** 새 버전 zip 올리기

**3** '비교하기' 누르기

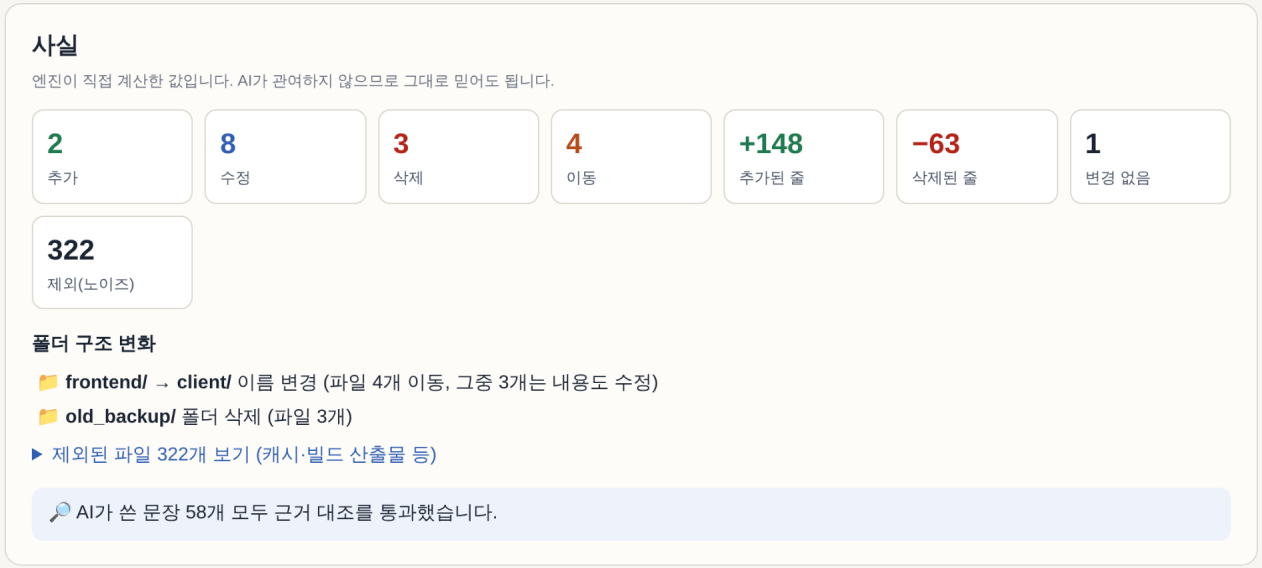
몇 분 뒤 결과 리포트가 화면에 열림. 그대로 내려받아 단톡방에 공유

설치할 것도, 배울 명령어도 없음 · Git을 몰라도 됨 · 압축 파일 그대로 넣으면 끝

실제로 써 보기 · 결과 리포트 맨 위

# 파일 330여 개 중, 진짜 바뀐 17개만 골라 준다

① 올리기 | ② 한눈에 보기 | ③ 근거 확인 | ④ 쉬운 말 보기 | ⑤ 경고 확인




**쓸데없는 파일은 알아서 뺀다**

자동으로 생기는 파일 322개를 빼고, 사람이 작업한 파일만 보여 줌

**폴더 이름을 바꿔도 안 헷갈린다**

'4개 삭제 + 4개 추가'가 아니라 '폴더 이름 변경'으로 알려 줌

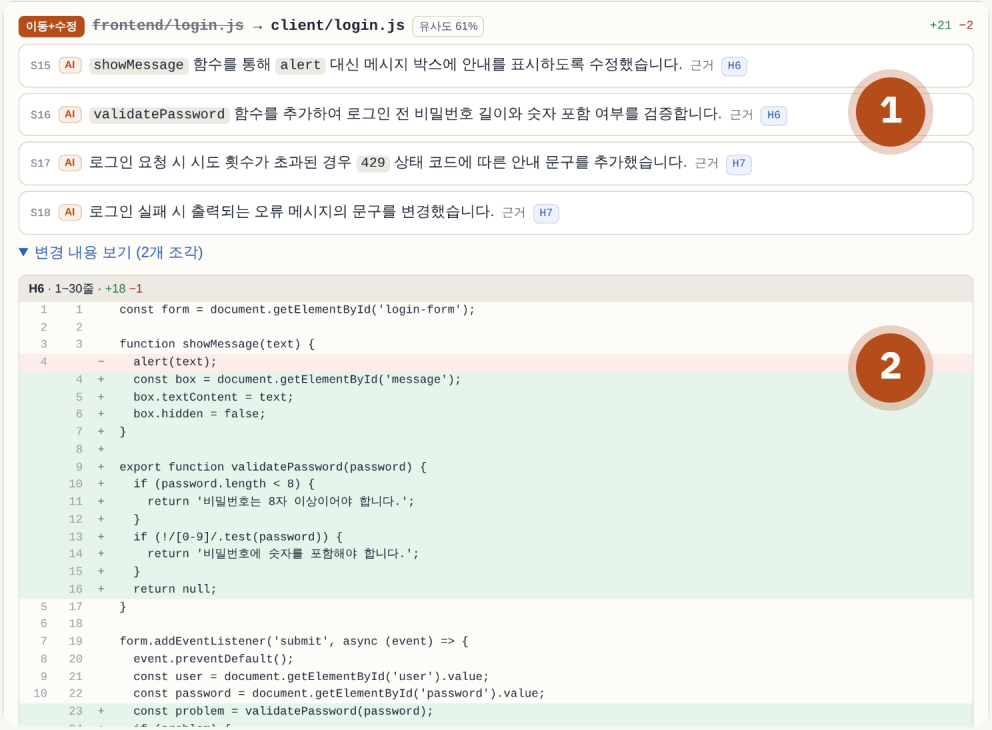
**숫자는 프로그램이 센다**

미리 만든 정답과 100% 일치

실제로 써 보기 · AI 설명 읽기

# AI 설명 옆 버튼을 누르면, 실제 바뀐 부분이 열린다

① 올리기 | ② 한눈에 보기 | ③ 근거 확인 | ④ 쉬운 말 보기 | ⑤ 경고 확인




### 1 AI 설명을 읽는다

"로그인 전에 비밀번호 길이와 숫자 포함 여부를 검사하는 기능이 추가됐다"

### 2 옆의 근거 버튼을 누른다

실제 바뀐 부분이 바로 펼쳐짐. 초록색이 새로 생긴 내용

### → 설명한 기능이 정말 있다

프로그램도 자동으로 한 번 더 대조

**AI 말을 그냥 믿지 않아도 된다**

팀 근성 · 중간보고 · 18 · 이번 실행에서 AI 설명 58문장 모두 실제 바뀐 부분과 연결

실제로 써 보기 · 코드를 모르는 팀원이라면

# 버튼 하나로, 쉬운 말 요약으로 바뀐다

① 올리기 | ② 한눈에 보기 | ③ 근거 확인 | ④ 쉬운 말 보기 | ⑤ 경고 확인




**1 '비개발자용'을 누른다**

함수 이름·파일 경로 없이 기능 중심으로 설명

**"회원가입 화면이 새로 생겼습니다"**

기획·디자인 담당도 진행 상황을 바로 이해

**궁금하면 '근거 보기'**

개발자용 화면의 그 위치로 바로 이동

팀 근성 · 중간보고 · 19 · 개선 중: 아직 '상태 코드' 같은 용어가 한 번 남아 있음

실제로 써 보기 · AI가 틀리면?

# '확인 필요' 표시가 보이면, 그 설명만 확인

① 올리기 | ② 한눈에 보기 | ③ 근거 확인 | ④ 쉬운 말 보기 | ⑤ 경고 확인

**실험** · AI 설명에 실제로는 없는 기능 이름과 엉터리 숫자를 일부러 섞어 봤더니

S15 AI `showMessage` 함수를 통해 `alert` 대신 메시지 박스에 안내를 표시하도록 수정했습니다. 근거 H6

S16 AI `validatePassword`, `checkEmailFormat` 함수를 추가하여 비밀번호를 12가지 규칙으로 검증합니다. 근거 H6 ⚠ 확인 필요

- **E3** `checkEmailFormat`가 근거 변경 내용에 없음
- **E4** 숫자가 포함됨 — 숫자 정보는 위쪽 '사실' 영역의 값을 확인하세요

S17 AI 로그인 요청 시 시도 횟수가 초과된 경우 `429` 상태 코드에 따른 안내 문구를 추가했습니다. 근거 H7

S18 AI 로그인 실패 시 출력되는 오류 메시지의 문구를 변경했습니다. 근거 H7

⚠ "그런 기능은 실제로 바뀐 내용에 없어요"

틀린 요약을 모르고 공유하는 일을 막는다

핵심 · AI와 프로그램의 경계

# AI는 문장만 쓴다. 숫자와 판정은 프로그램이 한다

### 프로그램

- 파일 읽기, 노이즈·비밀 파일 처리
- 추가·삭제·수정·이동 분류
- 줄 수·위치 등 모든 숫자
- 근거 번호(H) 부여
- AI 문장 자동 검증
- 리포트 생성

### AI (Gemini)

- 변경의 의미를 설명하는 문장
- 문장마다 근거 번호 선택
- 비개발자용 쉬운 설명

숫자를 쓰거나 근거가 틀리면 자동 경고

### 사람

- ⚠ 문장 근거 확인
- 의미가 맞는지 최종 판단

이번 시연 · 숫자·파일·위치 정보는 100% 프로그램이 계산 · AI 문장 58개는 모두 근거에 연결되어 검증 통과

진행 결과 · 실제로 도움이 된 것

# 그래서, 믿고 쓸 수 있는가

**무엇이 바뀌었는지 정확히**

## 97~100%

실제 공개 프로젝트의 4개 버전을 비교했을 때
전문가용 도구(Git)의 기록과 일치한 비율

**틀린 설명은 빠짐없이**

## 227 / 227

일부러 넣은 틀린 설명을 모두 경고
맞는 설명에 괜히 경고한 경우는 0번

**기다림 없이**

## 1초 미만

바뀐 파일 찾기와 확인까지
AI 설명은 파일 수에 따라 수 분

**솔직한 한계**

그럴듯하지만 **뜻만 살짝 틀린 설명**은 절반 정도만 잡힘
→ 그래서 근거 버튼으로 사람이 마지막에 확인

**다음 주에 확인할 것**

5명에게 직접 써 보게 해서
'직접 비교할 때보다 얼마나 빨라지는지' 측정 (11/12~18)

근거로 확인하는 AI 변경 요약 · 중간보고

# 감사합니다

남은 2주는 검증과 품질 개선에 집중